

Les principales API graphiques JAVA

Swing – SWT – JavaFX

Katy Saintin

Février 2016



- Historique des GUI à SOLEIL
- SWING
 - Architecture logicielle
 - Exemple de panneau
- SWT
 - Architecture logicielle
 - Exemple de panneau
 - Intégration de composant SWING
 - Exemple de projets
- JavaFX
 - Architecture logicielle
 - Exemple de panneau
 - Intégration de composant SWING

Ressources :

<http://www.bogotobogo.com/Java/tutorial/javagraphics.php>

http://www.bogotobogo.com/JavaAppletWebStart/awt_swing_swt.php

Le choix s'est porté sur SWING dès le départ :

- Fait partie du package JFC (Java Foundation Classes).
- AAK développé en SWING.
- Librairie pure Java.
- IDE propose des éditeurs (glisser/déplacer)
- GlobalSCREEN support des composants Swing.
- Il utilise le pattern MVC
- CometeSwing

Tentative d'application en SWT :

- CometeSWT en collaboration avec ANSTO (GumTree)
- DataBrowser SWT → document de faisabilité

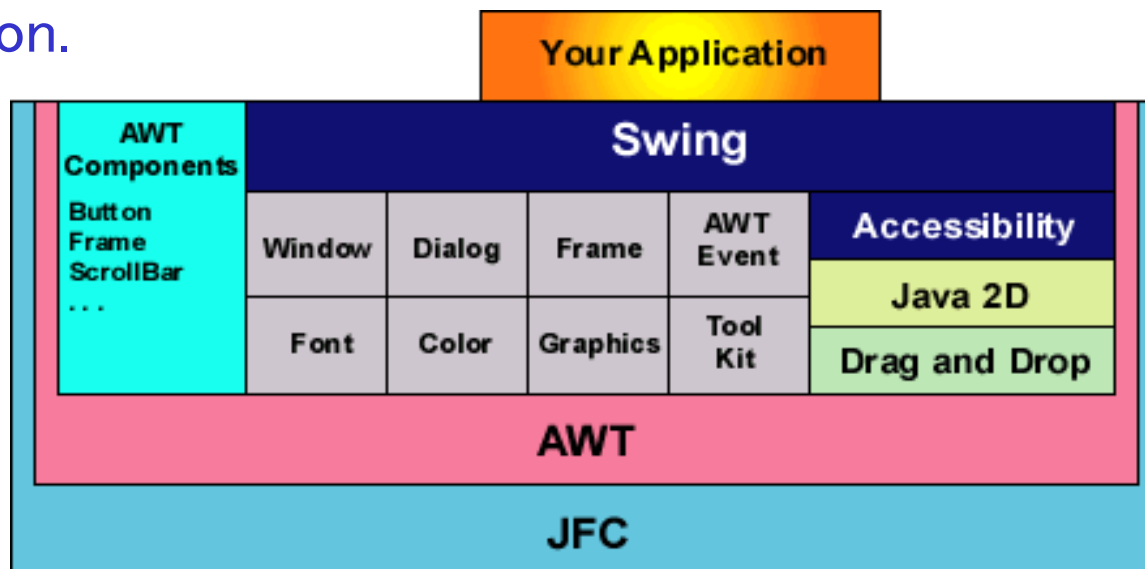
JavaFX en évaluation :

- Formation faite par Raphaël, Gregory et Katy en 2014
- Choisir une API pour la partie web ?
- CometeFX ?

AWT librairie indépendante du système d'exploitation. Chaque composant est dessiné (émulé).

SWING extension d' AWT utilise les mêmes objets de ressources qu' AWT (Color, Font ...) et le thread EDT

SwingX, extension.

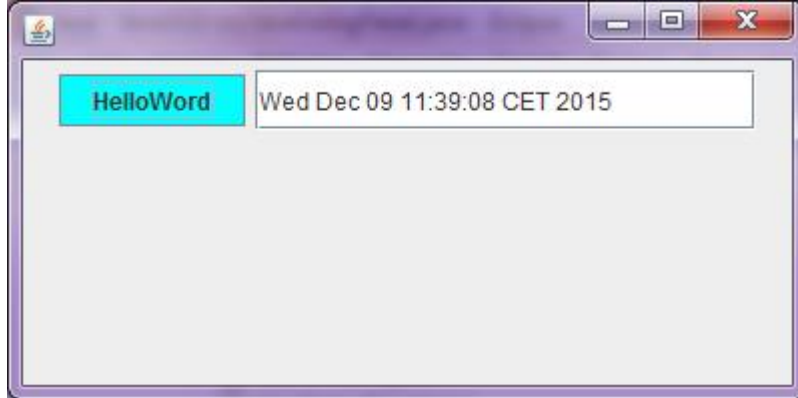


Swing est basé sur l'API AWT (Abstract Window Toolkit) :

- Les composants sont émulés par la JVM.
- Extensibles (SwingX and SwingLabs)
- Va être remplacé par JavaFX progressivement.

Thread EDT (event dispatching thread) :

- Unique Thread -> Java 7 SecondaryLoop
- Utilisation de l'EDT n'est pas obligatoire => des problèmes graphiques.
- Test du Thread : `SwingUtilities.isEventDispatchThread()`
- Exécution synchrone : `SwingUtilities.invokeAndWait(Runnable)`
- Exécution asynchrone : `SwingUtilities.invokeLater(Runnable)`



```
import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Date;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;
```

```
public class JavaSwingPanel extends JPanel {

    private JTextField jTextField;
    public JavaSwingPanel() {

        JButton button = new JButton("HelloWord");
        button.addActionListener(new ActionListener() {

            @Override
            public void actionPerformed(ActionEvent e) {
                System.out.println("HelloWord");
            }
        });

        button.setBackground(Color.CYAN);
        add(button);

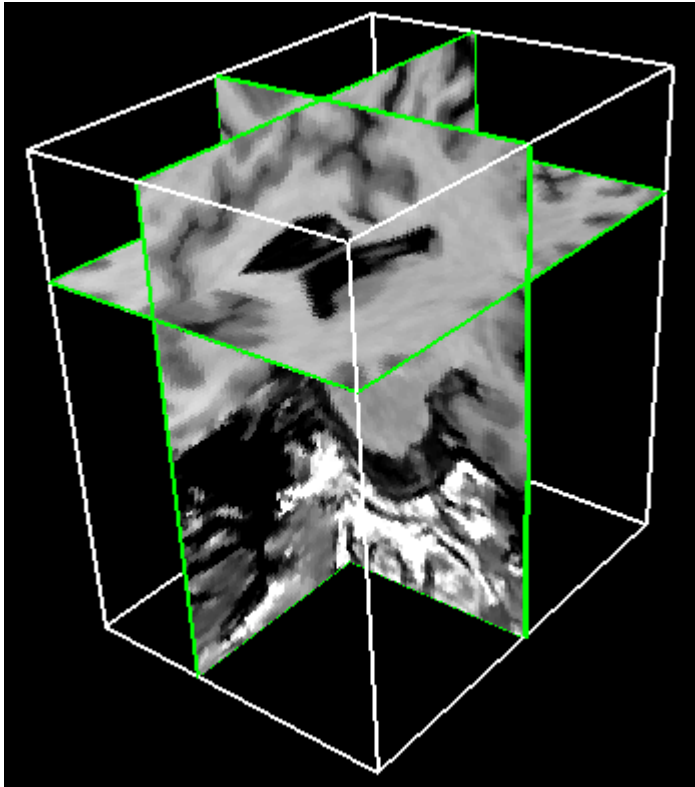
        jTextField = new JTextField();
        jTextField.setSize(250, 30);
        jTextField.setPreferredSize(jTextField.getSize());
        add(jTextField);
    }

    private void setText(String text) {
        jTextField.setText(text);
    }

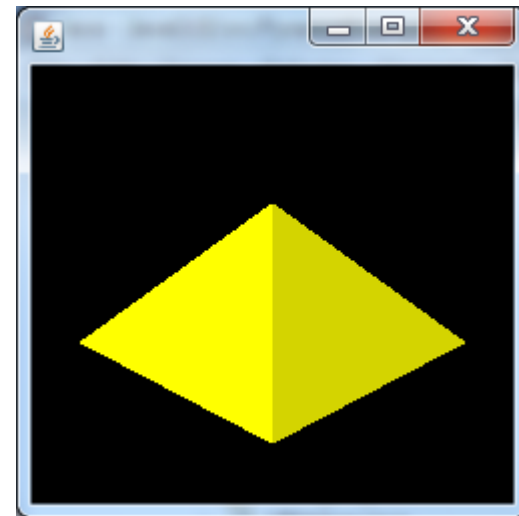
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        final JavaSwingPanel panel = new JavaSwingPanel();
        frame.setContentPane(panel);
        frame.setSize(400, 200);
        frame.setVisible(true);

        final Runnable setText = new Runnable(){
            @Override
            public void run() {
                panel.setText(new Date().toString());
            }
        };

        Thread thread = new Thread() {
            public void run() {
                while (true) {
                    SwingUtilities.invokeLater(setText);
                    try {
                        Thread.sleep(1000);
                    } catch (InterruptedException e) {
                        // TODO Auto-generated catch block
                        e.printStackTrace();
                    }
                }
            }
        };
    }
}
```



Basée sur **Java 3D**
Nécessite OpenGL ou DirectX et
dépend du système d'exploitation.



SWT (Standard Widget Toolkit), initiée par IBM mais n'est pas reconnu comme un standard Java. Implémentation native spécifique au système d'exploitation.

Utilisation

Eclipse utilise SWT
JFace extension.

Avantages

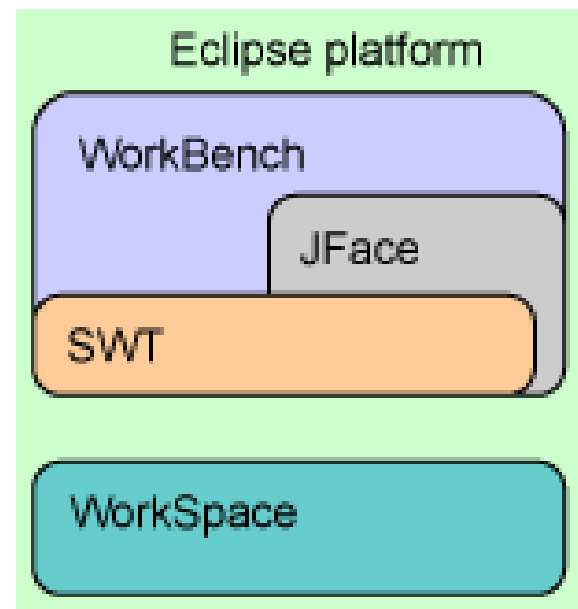
Plus rapide et mieux intégrée.

Inconvénients

Peu documentée.

Un comportement et look and feel dépendants de l'OS.

Gestion des ressources contraignante.



Thread SWT :

- Unique Thread
- Utilisation du Thread SWT est obligatoire au runtime.
- Test du Thread : `Display.getThread() == Thread.currentThread()`
- Exécution synchrone : `Display.syncExec(Runnable)`
- Exécution asynchrone : `Display.asyncExec(Runnable)`

```
import org.eclipse.swt.widgets.Display;
import org.eclipse.swt.widgets.Shell;
import org.eclipse.swt.widgets.Text;

public class JavaSWTPanel {

    public static void main (String [] args) {
        Display display = new Display ();
        Shell shell = new Shell(display);

        shell.setLayout(new FillLayout());
        shell.setSize(600, 100);

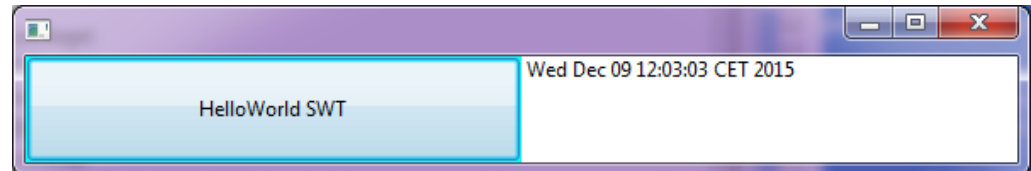
        Button button = new Button(shell,SWT.NONE);
        button.setText("HelloWorld SWT");
        Color color = new Color(display, 0,255,255);
        button.setBackground(color);
        //button.pack();
        Text helloWorldTest = new Text(shell, SWT.NONE);
        helloWorldTest.setText("Hello World SWT");
        //helloWorldTest.pack();

        final Runnable setText = new Runnable(){
            @Override
            public void run() {
                helloWorldTest.setText(new Date().toString());
            }
        };

        Thread thread = new Thread() {
            public void run() {
                while (true) {
                    display.asyncExec(setText);
                    try {
                        Thread.sleep(1000);
                    } catch (InterruptedException e) {
                        // TODO Auto-generated catch block
                        e.printStackTrace();
                    }
                }
            }
        };

        thread.start();

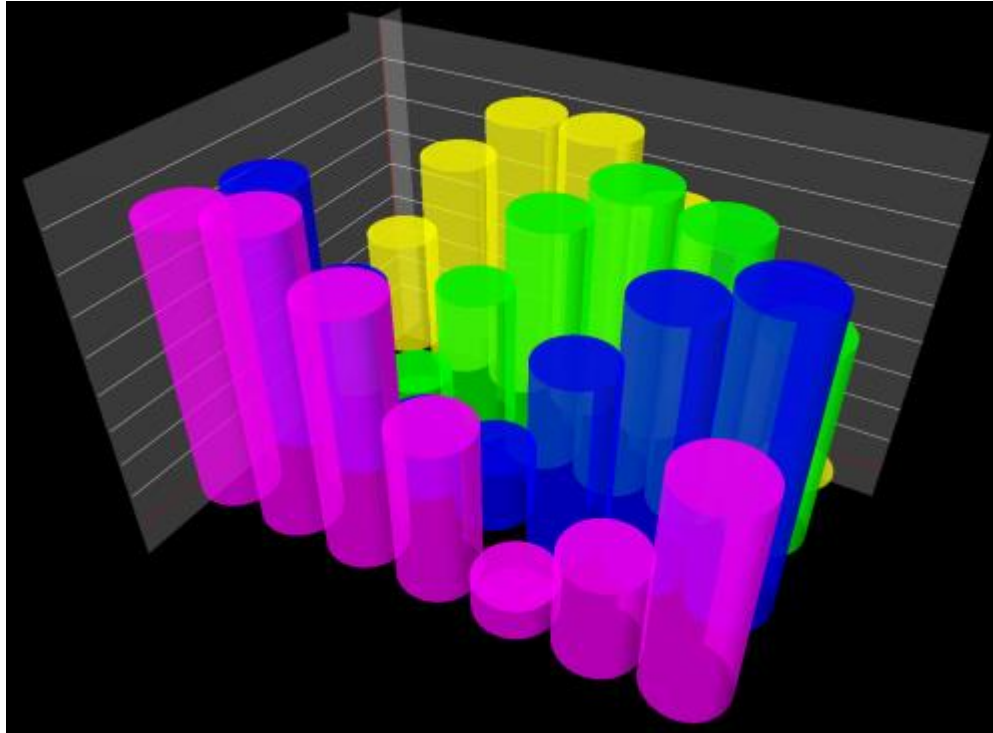
        shell.open ();
        while (!shell.isDisposed ()) {
            if (!display.readAndDispatch ()) display.sleep ();
        }
        display.dispose ();
    }
}
```

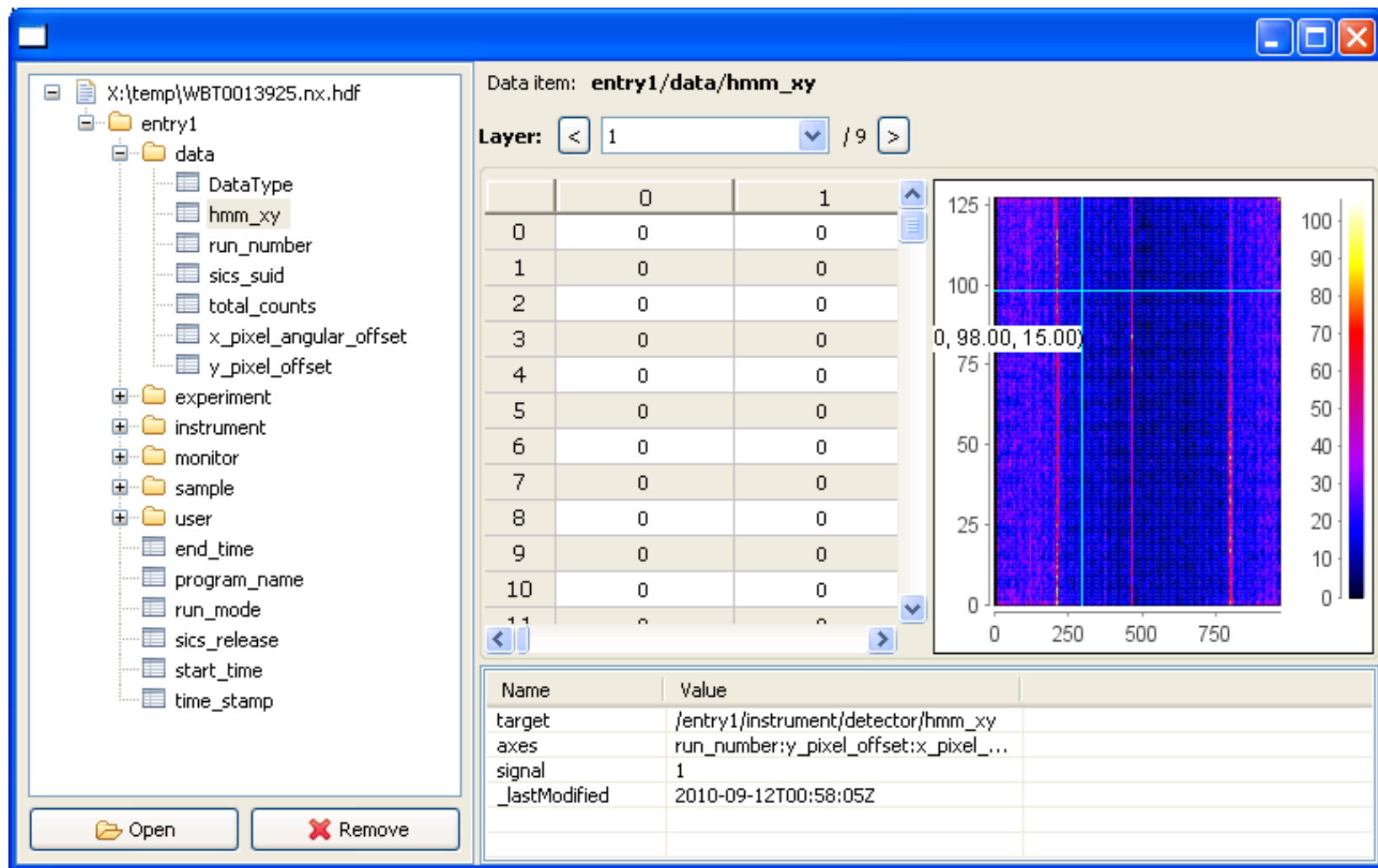


```
<terminated> JavaSWTPanel [Java Application] C:\Program Files\Java\jre1.8.0_65\bin\javaw.exe (9 d c. 2015 12:03:56)
Exception in thread "Thread-0" org.eclipse.swt.SWTException: Device is disposed
    at org.eclipse.swt.SWT.error(SWT.java:4083)
    at org.eclipse.swt.SWT.error(SWT.java:3998)
    at org.eclipse.swt.SWT.error(SWT.java:3969)
    at org.eclipse.swt.widgets.Display.error(Display.java:1249)
    at org.eclipse.swt.widgets.Display.asyncExec(Display.java:700)
    at JavaSWTPanel$2.run(JavaSWTPanel.java:43)
```

```
1 package swt;
2
3 import java.awt.Frame;
4
5 import javax.swing.JButton;
6
7 import org.eclipse.swt.SWT;
8 import org.eclipse.swt.awt.SWT_AWT;
9 import org.eclipse.swt.graphics.Color;
10 import org.eclipse.swt.layout.FillLayout;
11 import org.eclipse.swt.widgets.Button;
12 import org.eclipse.swt.widgets.Composite;
13 import org.eclipse.swt.widgets.Display;
14 import org.eclipse.swt.widgets.Shell;
15
16 public class SWTswingEmbeddedPanel {
17
18 public static void main (String [] args) {
19     final Display display = new Display ();
20     Shell shell = new Shell(display);
21
22     shell.setLayout(new FillLayout());
23     shell.setSize(600, 100);
24
25     Composite swingComp = new Composite(shell, SWT.EMBEDDED);
26     Frame new_Frame = SWT_AWT.new_Frame(swingComp);
27     JButton jButton = new JButton("Swing Button");
28     new_Frame.add(jButton);
29
30     Button button = new Button(shell, SWT.NONE);
31     button.setText("SWT Button");
32
33     Color color = new Color(display, 0,255,255);
34     button.setBackground(color);
35
36
37     shell.open ();
38     while (!shell.isDisposed ()) {
39         if (!display.readAndDispatch ()) display.sleep ();
40     }
41     display.dispose ();
42 }
43 }
44
```







Data item: **entry1/data/hmm_xy**

Layer: / 9

	0	1
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	0	0
8	0	0
9	0	0
10	0	0
11	0	0

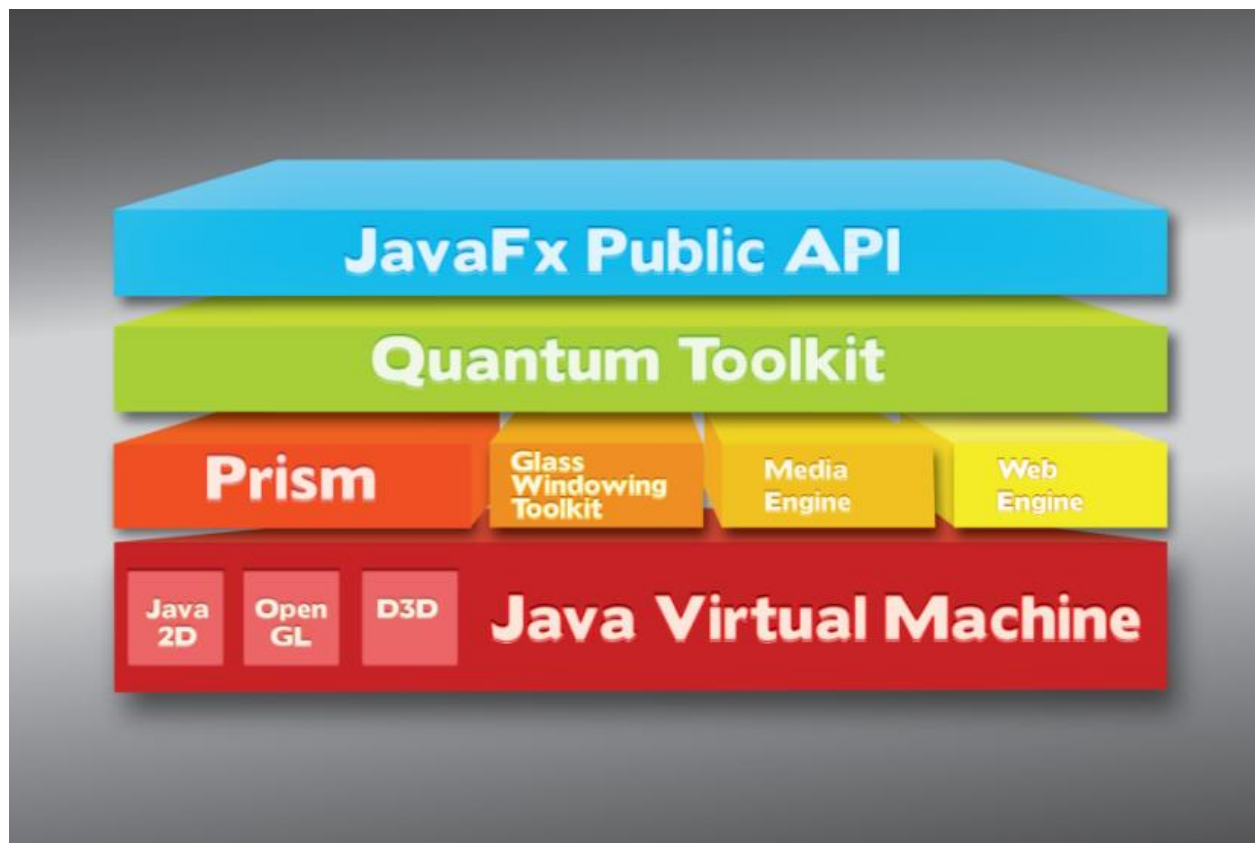
0, 98.00, 15.00

Name	Value
target	/entry1/instrument/detector/hmm_xy
axes	run_number:y_pixel_offset:x_pixel_...
signal	1
_lastModified	2010-09-12T00:58:05Z

Créé par Sun et appartient à Oracle maintenant.

Inclus dans Java 8 depuis mars 2014, nouveau standard.

Mixe SWING et XML(CSS)



Intégré dans les IDE NetBeans et Eclipse.

- JavaFX Script
- JavaFX Mobile

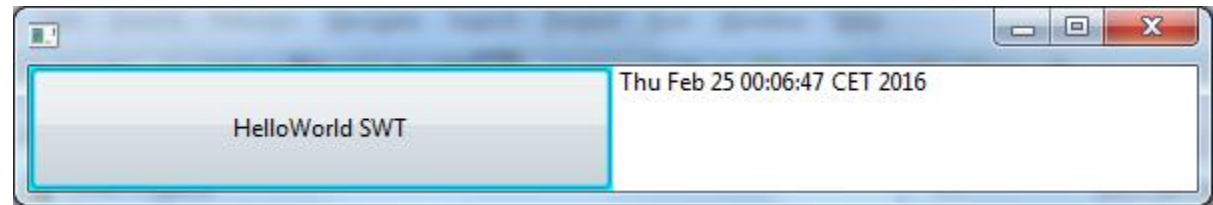
Thread JavaFX:

- Multi-tâche `javafx.concurrent.Task`; (Java 7)
- Utilisation du Thread JavaFX est obligatoire au runtime.
- Test du Thread : `isFxApplicationThread()`
- Exécution asynchrone : `Platform.runLater(setText);`

```

16 public class JavaFXPanel extends Application {
17     public static void main(String[] args) {
18         Launch(args);
19     }
20
21     @Override
22     public void start(Stage primaryStage) {
23         primaryStage.setTitle("Hello World!");
24         Button btn = new Button();
25         btn.setStyle("-fx-base:#00FFFF");
26         btn.setText("Say 'Hello World'");
27         btn.setOnAction(new EventHandler<ActionEvent>() {
28
29             @Override
30             public void handle(ActionEvent event) {
31                 System.out.println("Hello World!");
32             }
33         });
34
35         final Text textField = new Text();
36
37         FlowPane root = new FlowPane();
38         ObservableList<Node> children = root.getChildren();
39         children.add(btn);
40         children.add(textField);
41         primaryStage.setScene(new Scene(root, 300, 250));
42         primaryStage.show();
43
44         final Runnable setText = new Runnable(){
45             @Override
46             public void run() {
47                 textField.setText(new Date().toString());
48             }
49         };
50
51         Thread thread = new Thread() {
52             public void run() {
53                 while (true) {
54                     Platform.runLater(setText);
55                     try {
56                         Thread.sleep(1000);
57                     } catch (InterruptedException e) {
58                         // TODO Auto-generated catch block
59                         e.printStackTrace();
60                     }
61                 }
62             }
63         };
64
65         thread.start();
66     }
67 }

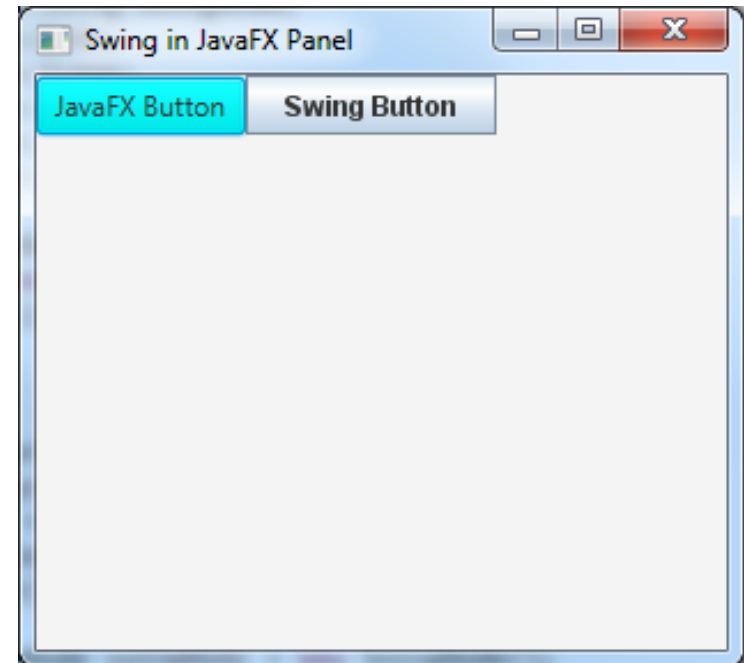
```




```

1 package javafx;
2 import javax.swing.JButton;
3 import javax.swing.SwingUtilities;
4
5 import javafx.application.Application;
6 import javafx.collections.ObservableList;
7 import javafx.embed.swing.SwingNode;
8 import javafx.scene.Node;
9 import javafx.scene.Scene;
10 import javafx.scene.control.Button;
11 import javafx.scene.layout.FlowPane;
12 import javafx.stage.Stage;
13
14 public class JavaFXSWINGEmbedded extends Application {
15     public static void main(String[] args) {
16         launch(args);
17     }
18
19     @Override
20     public void start(Stage primaryStage) {
21         primaryStage.setTitle("Swing in JavaFX Panel");
22         Button btn = new Button();
23         btn.setStyle("-fx-base:#00FFFF");
24         btn.setText("JavaFX Button");
25
26         final SwingNode swingNode = new SwingNode();
27         SwingUtilities.invokeLater(new Runnable() {
28             @Override
29             public void run() {
30                 swingNode.setContent(new JButton("Swing Button"));
31             }
32         });
33
34
35         FlowPane root = new FlowPane();
36         ObservableList<Node> children = root.getChildren();
37         children.add(btn);
38         children.add(swingNode);
39
40         primaryStage.setScene(new Scene(root, 300, 250));
41         primaryStage.show();
42
43
44     }

```



```

1 import java.awt.Color;
2 import javax.swing.JButton;
3 import javax.swing.JFrame;
4 import javax.swing.JPanel;
5 import javafx.collections.ObservableList;
6 import javafx.embed.swing.JFXPanel;
7 import javafx.scene.Node;
8 import javafx.scene.Scene;
9 import javafx.scene.control.Button;
10 import javafx.scene.layout.FlowPane;
11
12 public class JavaSwingEmbeddedFx extends JPanel {
13
14     public JavaSwingEmbeddedFx() {
15
16         JButton button = new JButton("SWING Button");
17         button.setBackground(Color.CYAN);
18         add(button);
19
20         final JFXPanel fxPanel = new JFXPanel();
21         add(fxPanel);
22
23         Button btn = new Button();
24         btn.setStyle("-fx-base:#00FFFF");
25         btn.setText("JavaFX Button");
26
27         FlowPane root = new FlowPane();
28         ObservableList<Node> children = root.getChildren();
29         children.add(btn);
30         Scene scene = new Scene(root, 100, 30);
31         fxPanel.setScene(scene);
32     }
33
34     public static void main(String[] args) {
35         JFrame frame = new JFrame();
36         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
37         final JavaSwingEmbeddedFx panel = new JavaSwingEmbeddedFx();
38         frame.setContentPane(panel);
39         frame.setSize(400, 200);
40         frame.setVisible(true);
41     }
42 }
43 }
44

```

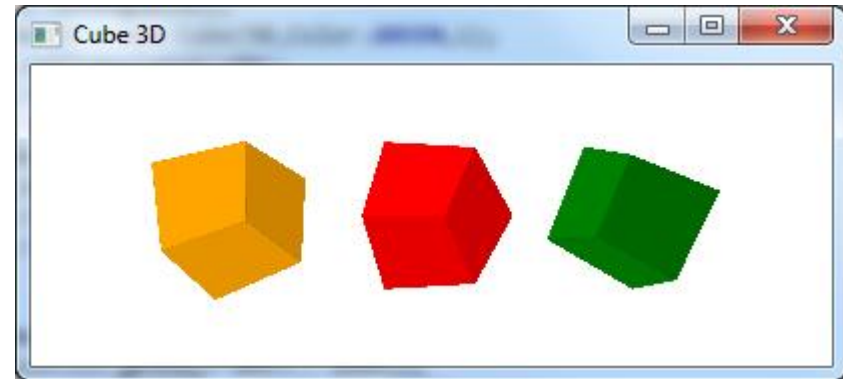


```
public class Cube3D extends Application{

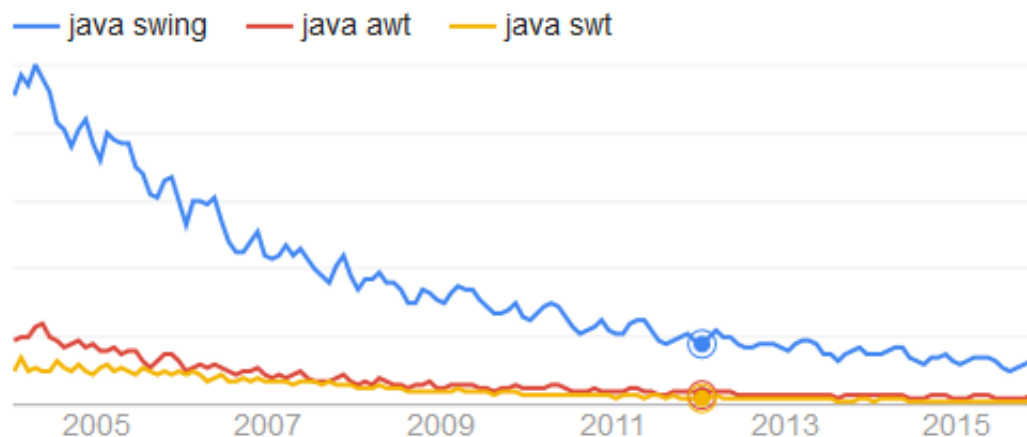
    @Override public void start(Stage stage) throws Exception {
        stage.setTitle("Cube 3D");
        //stage.initDepthBuffer(true);

        Cube c = new Cube(50,Color.RED,1);
        c.rx.setAngle(45);
        c.ry.setAngle(45);
        Cube c2 = new Cube(50,Color.GREEN,1);
        c2.setTranslateX(100);
        c2.rx.setAngle(45);
        c2.ry.setAngle(45);
        Cube c3 = new Cube(50,Color.ORANGE,1);
        c3.setTranslateX(-100);
        c3.rx.setAngle(45);
        c3.ry.setAngle(45);

        Timeline animation = new Timeline();
        animation.getKeyFrames().addAll(
            new KeyFrame(Duration.ZERO,
                new KeyValue(c.ry.angleProperty(), 0d),
                new KeyValue(c2.rx.angleProperty(), 0d),
                new KeyValue(c3.rz.angleProperty(), 0d)
            ),
            new KeyFrame(Duration.millis(1000),
                new KeyValue(c.ry.angleProperty(), 360d),
                new KeyValue(c2.rx.angleProperty(), 360d),
                new KeyValue(c3.rz.angleProperty(), 360d)
            )
        );
        animation.setCycleCount(Animation.INDEFINITE);
        // create root group
        Group root = new Group(c,c2,c3);
        // translate and rotate group so that origin is center and +Y is up
        root.setTranslateX(400/2);
        root.setTranslateY(150/2);
        root.getTransforms().add(new Rotate(180, Rotate.X_AXIS));
        // create scene
        Scene scene = new Scene(root, 400,150);
        scene.setCamera(new PerspectiveCamera());
        stage.setScene(scene);
        stage.show();
        // start spinning animation
    }
}
```



Interest over time. Web Search. Worldwide, 2004 - present.



Compare

Search terms ▾

javafx

Search term

java swing

Search term

+ Add term

Interest over time ?

☒ News headlines ☐ Forecast ?



</>

	Swing	SWT	JavaFX
Portabilité	Oui	Non(JNI)	Oui
Beau rendu	Oui avec Java 7 Nimbus	Oui	Oui
Web	Non (Applet)	Non	Oui
Thread safe	Non	Oui	Oui
Dynamisme	Oui	Non	–
Stabilité	Oui	Non	Oui
Documentation	Oui	Non	Oui
3D	Oui	Oui mais difficile à mettre en oeuvre	Oui

